Machine Learning for Finance

Supervised Learning - Introduction

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
   - Managing Many Features
   - Combining Predictions
5. Tips and Workflow

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
5. Tips and Workflow

Motivating with Momentum

- Suppose we want to predict returns with momentum
- Unlike unsupervised learning, we want to train our learning model on how to use momentum to make that prediction.
- The simplest supervised learner already in our toolbox is OLS; $r_i = \alpha + \beta Mom_i + u_i$

More generally,

$$Y_i = \alpha + \beta X_i + u_i$$

- Estimate via OLS; denote $\hat{\theta} = \{\hat{\alpha}, \hat{\beta}\}$
- X may be vector $(N \times 1)$ or a matrix $(N \times K)$
- X aka "features"; Y aka target
- If Y is continuous $\rightarrow$ regression. If dichotomous $\rightarrow$ classification
- Note: Regression and classification can be extended to a time series context, but we will eschew for now.

What is Supervised Learning?

Data: $(X_i, Y_i)_{i=1}^n$ drawn from joint distribution $P(X, Y)$.

Goal: Learn $f : \mathcal{X} \to \mathcal{Y}$ so predictions $\hat{Y} = f(X)$ are useful.

Population risk:

$$R(f) = \mathbb{E}_{(X,Y)\sim P}[L(Y, f(X))]$$

Empirical risk:

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} L(Y_i, f(X_i))$$

where L is some loss function and f is some functional form upon which we minimize empirical risk.

Functional Forms

Typical choices:

- Linear: $f(X) = \alpha + \beta^\top X$
- Trees, ensembles (RF, GBM)
- Neural nets
- k-NN, SVM

The more flexible the functional form, the more patterns it can fit.
Tradeoff: More flexibility can reduce bias, but increase variance.

Loss vs Evaluation Metric

- **Loss function (training):** A function $L(Y, f(X))$ minimized by the learning algorithm (often differentiable / convex). It drives parameter updates and defines the optimization objective.
- **Evaluation metric (reporting):** A performance measure used to compare models or to assess real-world usefulness. May be non-differentiable or domain-specific.
- **Key point:** *We optimize a loss; we evaluate with metrics.* They can differ (e.g., optimize log-loss but report AUC).
- **Practical implication:** Choose the loss to enable efficient learning and choose metrics that capture the business objective (e.g. calibration, ranking, cost-sensitive errors, etc...).

Common Loss Functions (used during training)

- **Regression**

$$L_{SE} = (Y - \hat{Y})^2 \quad \text{(squared error)}$$

$$L_{LA} = |Y - \hat{Y}| \quad \text{(absolute error)}$$

- **Classification / Probabilities**

$$L_{log} = -\big(Y \log p + (1 - Y) \log(1 - p)\big) \quad \text{(log)}$$

where p is the model's predicted probability that $Y = 1$

$$L_{hinge} = \max(0, 1 - y \cdot s(x)) \quad \text{(hinge)}$$

- **Other useful training losses:** focal loss (imbalanced classes), quantile loss (conditional quantiles).

Bias–Variance Decomposition (Squared Loss)

For squared error prediction at a point x:

$$\mathbb{E}\big[(Y - \hat{f}(x))^2\big] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

Interpretation:

- Complex models: $\downarrow$ bias, $\uparrow$ variance
- Simple models: $\uparrow$ bias, $\downarrow$ variance
- σ^2 is randomness in Y — even the true model cannot remove it

Goal: Select model complexity to balance bias and variance, then fit parameters to minimize training loss

Evaluation Metrics

Regression metrics

- RMSE: $\sqrt{\frac{1}{N}\sum(Y-\hat{Y})^2}$ — penalizes large errors
- MAE: $\frac{1}{N}\sum|Y-\hat{Y}|$ — robust to outliers
- R^2 (explained variance): $1-\frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$

Classification metrics

- Accuracy, Precision, Recall, F1 (threshold-dependent)
- AUC / ROC (threshold-free ranking metric)
- Brier score (calibration of probabilistic forecasts)
- Log-loss (evaluates probabilistic accuracy)

Error Diagnostics

- Residual vs fitted / feature plots: look for systematic patterns, which could signal misspecification
- QQ-plot of residuals: check distributional assumptions (if relevant)
- Heteroscedasticity check: residuals^2 vs features
- Confusion matrix & cost analysis (i.e. FP, FN, etc..) for operational decisions (use domain-specific costs)

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
5. Tips and Workflow

Why Not Evaluate on the Same 500 Assets?

Setting: 500 assets at time t. We use firm characteristics X_i to predict returns Y_i.
We estimate:

$$\hat{Y}_i = \hat{\alpha} + \hat{\beta} X_i \quad \text{using all 500 assets.}$$

Then we compute RMSE using those same 500 assets and take long positions in the assets with the highest expected returns and short positions in the lowest expected returns.

Q: Why is this not a reliable estimate of investment performance?
A:

- The model was chosen to fit these exact 500 observations
- If we allow enough flexibility, we can drive training error arbitrarily close to zero
- But that does not mean we can predict new data well

A Solution: Train / Test Splitting

Setting: 500 assets at time t. We use firm characteristics X_i to predict returns Y_i.

Step 1: Split the sample

- Training sample: randomly select 80% of assets (e.g., 400 firms)
- Test sample: use the remaining 20% of assets (e.g., 100 firms)

Step 2: Estimate model on training data

$$\hat{Y}_i^{Train} = \hat{\alpha}^{Train} + \hat{\beta}^{Train} X_i \quad \text{for } i \in \text{Training set}$$

Step 3: Evaluate on test data

$$\hat{Y}_j^{Test} = \hat{\alpha}^{Train} + \hat{\beta}^{Train} X_j \quad \text{for } j \in \text{Test set}$$

$$RMSE_{Test} = \sqrt{\frac{1}{N_{Test}} \sum_{j \in Test} (Y_j - \hat{Y}_j^{Test})^2}$$

Test performance estimates generalization error.

Train/Validation/Test

Many supervised models have **hyperparameters** τ:

- Ridge penalty λ
- Tree depth
- Number of boosting rounds
- Neural network architecture

These are not learned automatically — we must choose them. If we tune hyperparameters using the test set, we contaminate the test set.

Solution:

- Training set: estimate parameters θ
- Validation set: choose hyperparameters τ
- Test set: final unbiased evaluation

The Problem With Single Splits

Q: What's the problem with use a single train, single validate, single test?

A single train/validation/test split depends on:

- Which observations were held out, and
- ...random sampling variation, which can ....
- ...cause the test error estimate can have high variance.

Solution: Cross-validation, which reduces variance by averaging across multiple splits.

K-Fold Cross-Validation

Repeatedly hold out data

Example: 500 assets in training set. Choose $K = 5$

- Split data into 5 equal groups ("folds") of 100 assets each.
- A **fold** is simply one subset used as temporary validation data.

For $k = 1, \ldots, 5$:

- Train the model on 4 folds (400 assets)
- Compute validation error on the held-out fold (100 assets)
- Record validation error R_k

Cross-validated error:

$$\hat{R}_{CV} = \frac{1}{K} \sum_{k=1}^{K} R_k$$

Use $\hat{R}_{CV}$ to compare models or tune hyperparameters.

How Do We Choose K?

Choosing K controls the bias–variance tradeoff of the cross-validation error estimate. Q: If K is small, is bias higher/lower? Is variance higher/lower?

Small K (e.g., 2–3)

- Larger validation sets
- Higher bias (model trained on less data)
- Lower variance

Large K (e.g., Setting $K = N$ is referred to LOOCV, Leave One Out CV)

- Larger training sets
- Low bias
- High variance
- Computationally expensive

Hyperparameter Tuning with CV

Step 1: Split data into Training and Test

Step 2: Use K-fold cross-validation on the Training set, where each fold temporarily acts as a validation set.

Algorithm:

1. Define hyperparameter grid $\{\tau\}$
2. For each τ, compute cross-validated risk $\hat{R}_{CV}(\tau)$
3. Select $\tau^* = \arg\min \hat{R}_{CV}(\tau)$

Step 3: Refit model on full Training set using τ^*

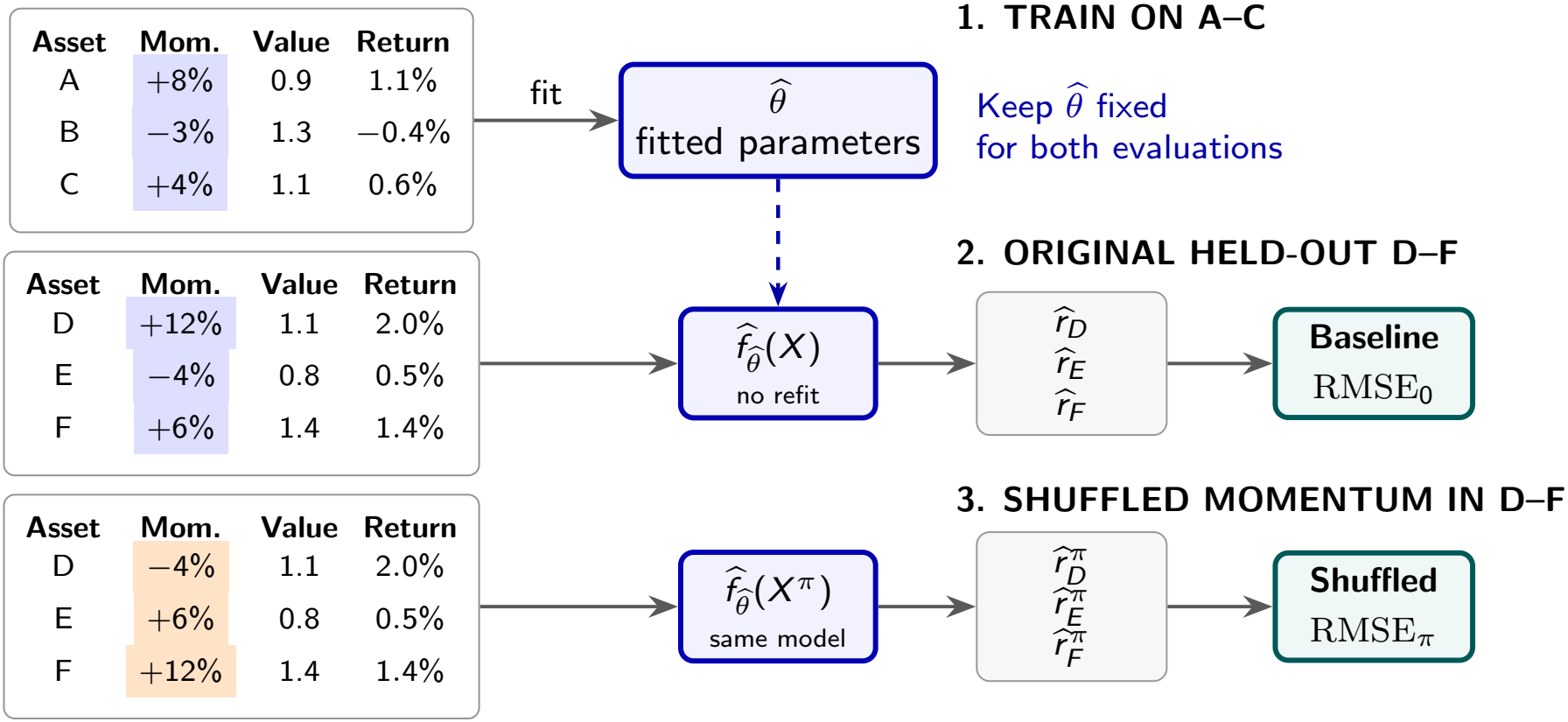
Step 4: Evaluate once on Test set

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
5. Tips and Workflow

Permutation Importance: Fit Once, Evaluate Twice

Goal: Measure how much predictive accuracy deteriorates when we disrupt Momentum, *without refitting the model*.



Notice: D, E, F retain their Value and realized returns; only their Momentum assignments change.

Permutation Importance: Quantifying Predictive Reliance

Define two errors on the same held-out assets:

$$\text{Original:} \quad \widehat{r}_i = \widehat{f}_{\widehat{\theta}}(x_{i,\text{Mom}}, x_{i,-\text{Mom}}), \qquad L_0 = \text{RMSE}(r, \widehat{r}),$$

$$\text{Shuffled:} \quad \widehat{r}_i^{\pi} = \widehat{f}_{\widehat{\theta}}(x_{\pi(i),\text{Mom}}, x_{i,-\text{Mom}}), \qquad L_\pi = \text{RMSE}(r, \widehat{r}^{\pi}).$$

Repeat with B random permutations:

$$\boxed{PI_{\text{Mom}} = \frac{1}{B}\sum_{b=1}^{B} L_{\pi_b} - L_0}$$

Illustration: If baseline RMSE is 1.2% and mean shuffled RMSE is 1.7%, $PI_{\text{Mom}} = 1.7\% - 1.2\% = \mathbf{0.5}$ percentage points. Disrupting Momentum increases out-of-sample prediction error, indicating that the fitted model relies on Momentum for predictive accuracy.

- Caution: When predictors are correlated, shuffling can create implausible feature combinations.

SHAP: A Regression-Based Intuition

Goal: Allocate one asset's *fitted prediction* among its features, relative to a baseline.

Perspective: Read each path left to right: hypothetically reveal feature values one at a time and measure the change in the prediction.

Fit once: $\widehat{f}(x_1, x_2) = \widehat{a} + \widehat{b}_1 x_1 + \widehat{b}_2 x_2 + \widehat{\gamma} x_1 x_2$. **Ordering A:** $X_1 \to X_2$

$$\widehat{a} \xrightarrow{x_1} \widehat{a} + \widehat{b}_1 x_1$$

$$\Delta_1^{(A)} = (\widehat{a} + \widehat{b}_1 x_1) - \widehat{a} = \widehat{b}_1 x_1$$

Ordering B: $X_2 \to X_1$

$$\widehat{a} + \widehat{b}_2 x_2 \xrightarrow{x_1} \widehat{a} + \widehat{b}_2 x_2 + \widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2$$

$$\begin{aligned} \Delta_1^{(B)} &= \left(\widehat{a} + \widehat{b}_2 x_2 + \widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2\right) \\ &\quad - \left(\widehat{a} + \widehat{b}_2 x_2\right) \\ &= \widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2 \end{aligned}$$

SHAP gives the two paths equal weight: $\phi_1(x) = \frac{1}{2}\Delta_1^{(A)} + \frac{1}{2}\Delta_1^{(B)} = \widehat{b}_1 x_1 + \frac{1}{2}\widehat{\gamma} x_1 x_2$.

No refitting: Same fitted coefficients throughout. Unknown features are averaged over an independent, zero-mean background.

SHAP: Three Features and Six Orderings

Now consider three features:

$$\widehat{f}(x_1, x_2, x_3) = \widehat{a} + \widehat{b}_1 x_1 + \widehat{b}_2 x_2 + \widehat{b}_3 x_3 + \widehat{\gamma} x_1 x_2$$

Attribute to X_1: Average across all $3! = 6$ possible orderings.

Sequence of information	**Known before X_1**	**Change when X_1 is revealed**
$X_1 \to X_2 \to X_3$	None	$\widehat{b}_1 x_1$
$X_1 \to X_3 \to X_2$	None	$\widehat{b}_1 x_1$
$X_3 \to X_1 \to X_2$	X_3	$\widehat{b}_1 x_1$
$X_2 \to X_1 \to X_3$	X_2	$\widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2$
$X_2 \to X_3 \to X_1$	X_2, X_3	$\widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2$
$X_3 \to X_2 \to X_1$	X_2, X_3	$\widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2$

Equal weight for each sequence:

$$\phi_1(x) = \frac{1}{6}\left[3\widehat{b}_1 x_1 + 3\left(\widehat{b}_1 x_1 + \widehat{\gamma} x_1 x_2\right)\right] = \boxed{\widehat{b}_1 x_1 + \tfrac{1}{2}\widehat{\gamma} x_1 x_2}$$

Grouping equivalent comparisons gives the weights: $\frac{2}{6}$ (none), $\frac{1}{6}$ (X_2), $\frac{1}{6}$ (X_3), $\frac{2}{6}$ (X_2, X_3).

No refitting. Unknown features are averaged over the same independent, zero-mean background.

SHAP: Mathematics

Goal: Allocate an individual model prediction among features, relative to a background baseline. **Shapley attribution:**

$$\phi_j(x_i) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[v_{x_i}(S \cup \{j\}) - v_{x_i}(S) \right]$$

- F: full feature set; S: subset excluding feature j.
- $v_x(S)$: prediction averaged over features not in S, under a *specified background/dependence convention*.
- $\phi_j(x)$: allocated contribution of feature j to this asset's prediction.
- **Additivity:** $f(x) = \phi_0 + \sum_{j=1}^{p} \phi_j(x)$ (on the model-output scale being explained).

Feature correlation and the background distribution can alter attribution; SHAP is explanatory, not causal.

SHAP Example: Explaining Cross-Sectional Fit

Setting: At date t, predict asset returns from Momentum, Size, Value, and Volatility. For Asset A, the fitted return is $\hat{r}_A = 2.5\%$.

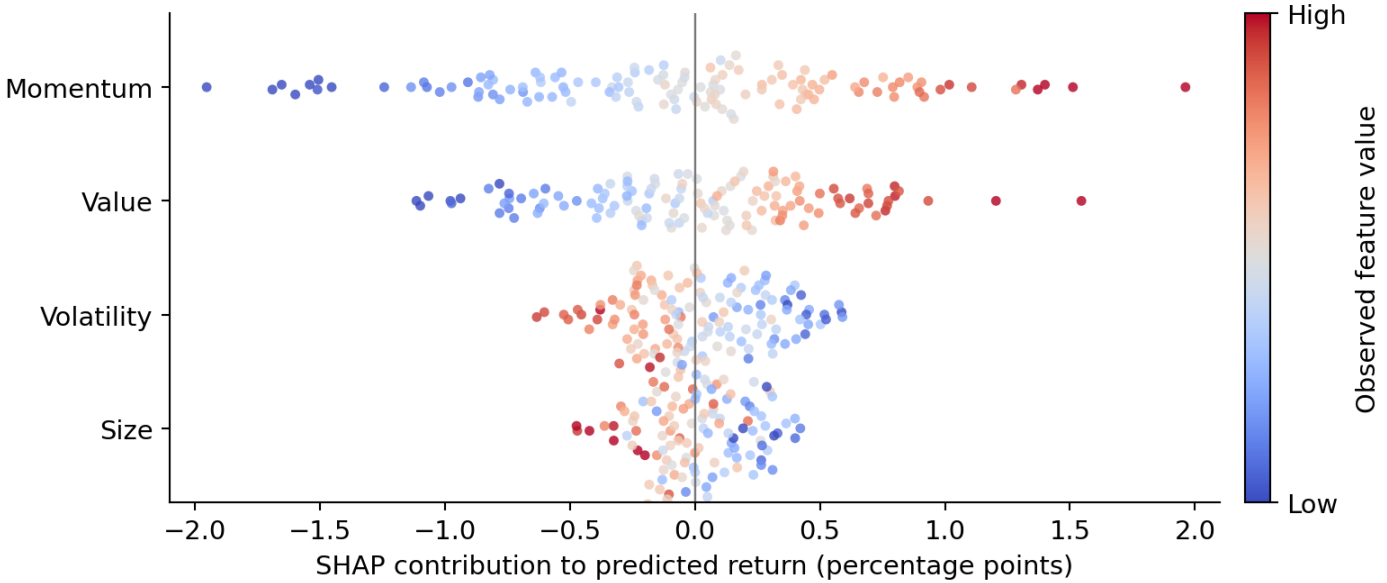
Baseline predicted return: $\phi_0 = 1.0\%$ (mean model output in a background sample).

$$\underbrace{2.5\%}_{\text{Predicted return}} = \underbrace{1.0\%}_{\text{Baseline}} + \underbrace{0.9\%}_{\text{Momentum}} \underbrace{-0.2\%}_{\text{Size}} + \underbrace{0.6\%}_{\text{Value}} + \underbrace{0.2\%}_{\text{Volatility}}$$

- Momentum, Value, and Volatility push the prediction *above* the baseline.
- Size contributes -0.2 percentage points *relative to the baseline*.
- Contributions sum to the fitted value; they need not be causal effects.

Summarizing SHAP

We can summarize the local SHAP values across assets for each feature ala $|\bar{\phi}_j| = \frac{1}{N}\sum_{i=1}^{N}|\phi_j(x_i)|$, or depict their distribution via beeswarm:



- Each dot: one asset's attribution; horizontal location: sign and size of contribution.
- Red/blue: higher/lower *observed* feature value; rows typically ordered by mean $|\mathrm{SHAP}|$.
- Reveals heterogeneity and direction that a single importance bar cannot show.

Schematic synthetic values; not results estimated from the 500-asset example.

Partial Dependence: From Data to One Point

Goal: Describe the fitted model's *average predicted return* as Momentum varies.

Step 1. Fit on full training data: $\widehat{r}_i = \widehat{f}_{\widehat{\theta}}(\mathrm{Mom}_i, \mathrm{Size}_i, \mathrm{Value}_i, \mathrm{Vol}_i)$; keep $\widehat{\theta}$ fixed.

Observed characteristics (3 rows shown)

	Mom.	Size	Value	Vol.
A	−2.0	−1.0	0.8	0.2
B	0.0	0.0	1.3	0.7
C	2.0	1.0	0.6	1.1

estimate →

Original fitted model

$\widehat{f}_{\widehat{\theta}}$

Parameters unchanged

Step 2. Replace every Momentum with $z = +1$

	Mom.	Size	Value	Vol.
A	**+1**	−1.0	0.8	0.2
B	**+1**	0.0	1.3	0.7
C	**+1**	1.0	0.6	1.1

same model →

Step 3. Predict for all assets

$\widehat{r}_A(+1) = 0.4\%$

$\widehat{r}_B(+1) = 0.8\%$

$\widehat{r}_C(+1) = 1.2\%$

Step 4. Average: $\widehat{PD}_{\mathrm{Mom}}(+1) = (0.4 + 0.8 + 1.2)/3 = \mathbf{0.8\%}$

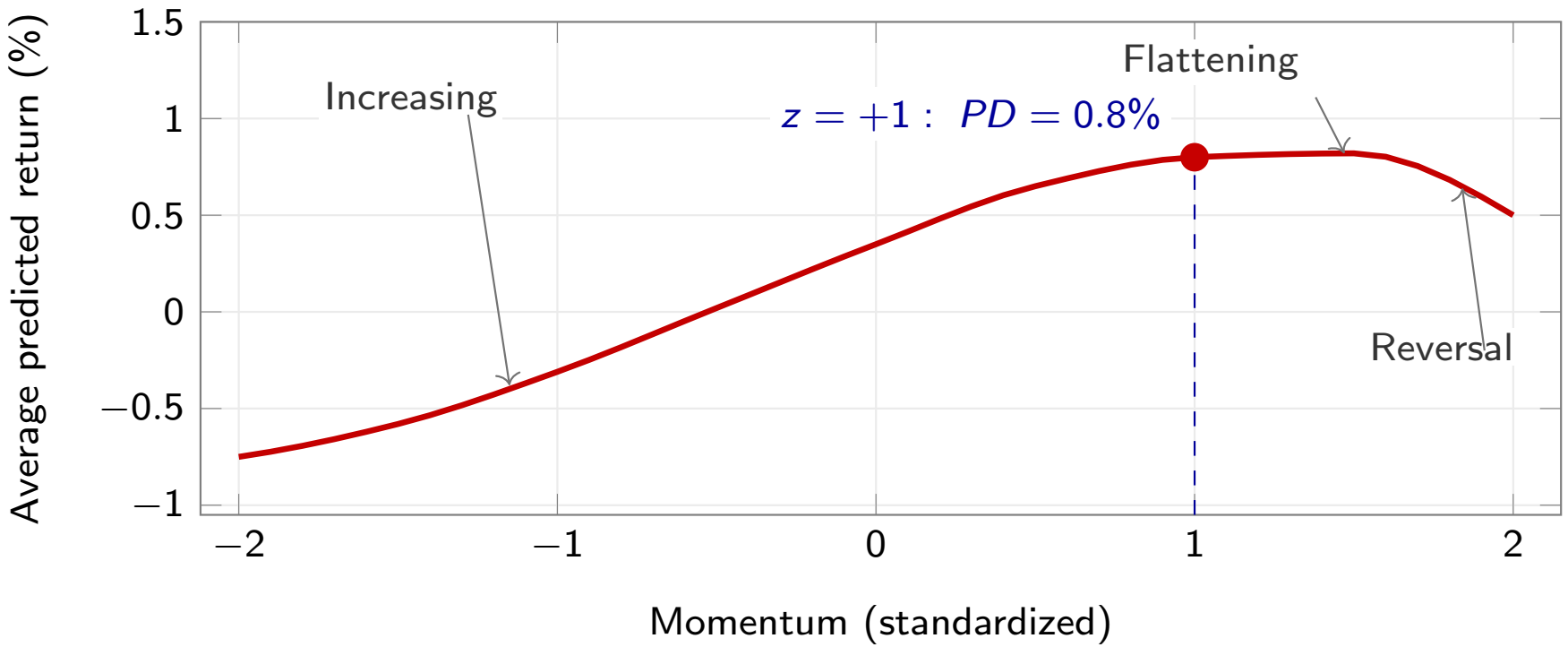
Repeat over a grid of z values within the observed range; average predictions at each z.

$$\widehat{PD}_{\mathrm{Mom}}(z) = \frac{1}{N}\sum_{i=1}^{N} \widehat{f}_{\widehat{\theta}}(z, x_{i,-\mathrm{Mom}})$$

$x_{i,-\mathrm{Mom}}$ denotes asset i's Size, Value, and Volatility.

Partial Dependence: Reading a Nonlinear Response

Each grid value z produces one average prediction. Connecting those points gives the PDP.



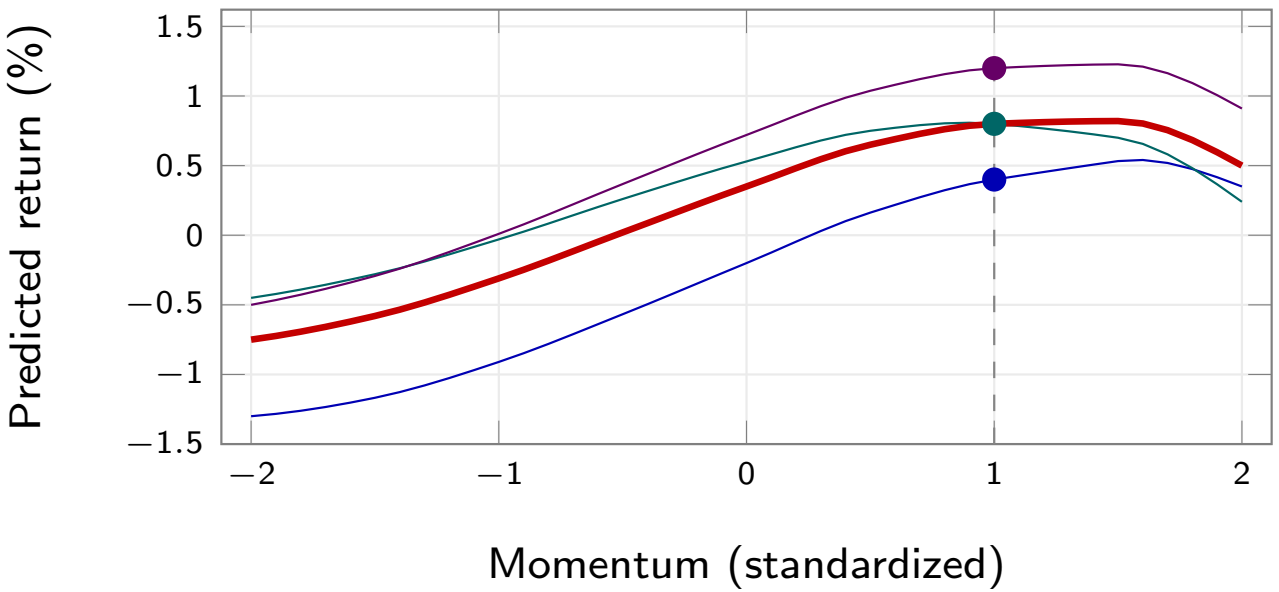
- The fitted model predicts a rising, then flattening and eventually declining *average* return as Momentum increases.
- **But an average can hide opposing responses across assets.**

Hypothetical nonlinear response, not an empirical finding or a causal effect; sparse or correlated-feature combinations merit caution.

ICE: What Does the PDP Average Hide?

Goal: Retain each asset's model-implied response as Momentum varies, rather than average immediately.

$$\widehat{ICE}_{i,\mathrm{Mom}}(z) = \widehat{f}_{\widehat{\theta}}(z, x_{i,-\mathrm{Mom}}), \qquad \widehat{PD}_{\mathrm{Mom}}(z) = \frac{1}{N}\sum_{i=1}^{N} \widehat{ICE}_{i,\mathrm{Mom}}(z).$$



At $z = +1$:

$\widehat{r}_A = 0.4\%$

$\widehat{r}_B = 0.8\%$

$\widehat{r}_C = 1.2\%$

PDP $= 0.8\%$

Thick line: average (PDP)

Thin lines: assets A–C (ICE)

- Different slopes and turning points show *heterogeneous modeled responses*, which the PDP can conceal.

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
   - Managing Many Features
   - Combining Predictions
5. Tips and Workflow

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
   - Managing Many Features
   - Combining Predictions
5. Tips and Workflow

Many Features

Q: Advantage of many features?
A: Potentially lower bias / better predictions
Q: Disadvantage of many features?
A:

- Large K expands hypothesis space and variance
- Linear models: $\hat{\beta} = (X'X)^{-1}X'y$ becomes unstable when $X'X$ ill-conditioned
- Nonlinear models: more candidate splits/parameters $\rightarrow$ more opportunity to fit noise

How to Handle Many Features

Main strategies:

1. Dimension reduction (PCA, autoencoders, supervised screening)
2. Regularization (Ridge, Lasso, Elastic Net)

Regularization (Ridge / Lasso / Elastic Net)

Ridge (L2):

$$\hat{\beta}^{ridge} = \arg\min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

- Shrinks coefficients toward zero.
- Rarely sets coefficients exactly to zero.
- Stabilizes estimates under multicollinearity.

Lasso (L1):

$$\hat{\beta}^{lasso} = \arg\min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

- Shrinks and can set coefficients exactly to zero.
- Performs automatic variable selection.
- Can be unstable when features are highly correlated.

Regularization (Ridge / Lasso / Elastic Net)

Elastic Net:

$$\hat{\beta}^{EN} = \arg\min_{\beta} \frac{1}{2n}\|y - X\beta\|_2^2 + \lambda(\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|_2^2)$$

Combines shrinkage and variable selection; useful when predictors are correlated.

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
   - Managing Many Features
   - Combining Predictions
5. Tips and Workflow

Model Averaging and Bagging

Model Averaging: Combine predictions from multiple models:

$$\hat{y}(x) = \sum_{m=1}^{M} w_m \hat{f}_m(x)$$

Averaging reduces variance when models make different errors.

Bagging (Bootstrap Aggregating):

- Draw B bootstrap samples from the training data.
- A **bootstrap sample** is drawn with replacement.
- Fit a model on each sample: $\hat{f}_b(x)$.
- Average predictions:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x)$$

Bagging primarily reduces variance. e.g. Random Forest

Boosting and Stacking

Boosting: Sequentially fit weak learners to residuals:

$$f_m(x) = f_{m-1}(x) + \nu h_m(x)$$

Each learner focuses on correcting previous mistakes.

- $h_m(x)$: the m-th weak learner fit to current residuals
- ν: learning rate (shrinks each learner's contribution)

Examples: AdaBoost, Gradient Boosting, XGBoost.

Stacking:
Train multiple base learners and combine via a meta-learner:

$$\hat{y}(x) = g(\hat{f}_1(x), \ldots, \hat{f}_M(x))$$

Super Learner

- CV-based ensemble that learns weights to minimize the prediction loss of the *combined* model.

$$\hat{\alpha} = \arg\min_{\alpha} \frac{1}{n} \sum_{i=1}^{n} L\left(Y_i, \sum_{m=1}^{M} \alpha_m \hat{f}_m^{(-v(i))}(X_i)\right)$$

$$\text{s.t.} \quad \alpha_m \geq 0, \qquad \sum_{m=1}^{M} \alpha_m = 1$$

  - $\hat{f}_m^{(-v(i))}(X_i)$: prediction from model m, trained without the validation fold containing i.
  - α_m: weight assigned to model m.

- Under suitable conditions, asymptotically performs at least as well as the best candidate in the library.

Outline

1. Foundations: Problem, Loss, Risk
2. Train / Validate / Test & Cross-Validation
3. Feature Importance & Interpretability
4. Model Complexity and Ensemble Learning
5. Tips and Workflow

Data Preprocessing aka Feature Engineering

- Standardize/normalize features (learn scaling on training only).
- Encode categorical variables (one-hot, target encoding — with care).
- Impute or model missingness (use training folds for imputation).
- Think carefully about interactions
- Be sure to apply transformations consistently across features

Data Leakage (Pitfalls)

Data leakage = test information used during training/tuning.

Examples:

- Scaling using full dataset before split.
- Feature selection (or imputation) that used test labels.
- Using future information for features in a forecasting context.

Result: over-optimistic performance and models that fail in production.

Recommended Practical Workflow

1. Define problem, loss, and metric.
2. Split data: train / validate (CV) / test.
3. Preprocess using training folds only.
4. Baseline with simple model (OLS, logistic).
5. Perform feature engineering and selection inside CV.
6. Tune hyperparameters via CV.
7. Evaluate on held-out test set.
8. Inspect interpretability (SHAP, PDP), calibration, and robustness (stress tests).